Jose Fabian

CS 499: Computer Science Capstone

Milestone Four: Enhancement Three — Databases Narrative

Artifact Description

The artifact for this enhancement is the CRUD module from my Grazioso Salvare Animal Shelter Dashboard, originally created for CS-340 (Client/Server Development). The artifact is a Python class, AnimalShelter, that manages the connection to a MongoDB database and provides the create, read, update, and delete operations the dashboard uses to work with animal records. The single file enhanced in this milestone is crud_module.py, which the dashboard depends on for all of its data access.

Justification for Inclusion

I selected this artifact for the databases category because it is the data-access layer of a real full-stack application, which means the quality of this one file directly determines whether the entire dashboard works, fails cleanly, or fails dangerously. It gave me the chance to demonstrate database connection management, secure credential handling, complete CRUD design, and query performance, all in a focused piece of code. The components that showcase my database skills are the MongoDB connection setup, the four CRUD methods, and the indexing strategy for the fields the dashboard queries most.

The artifact was improved in several concrete ways, and the starting point was unusually clear because the original file did not actually run. During the Milestone One code review, I found that the file had been corrupted: a stray shell heredoc command had been appended after the read

method, which meant the module could not even be imported without raising a SyntaxError. The first and most important fix was simply removing that corrupted content and restoring the file to valid Python. Beyond that, I completed the CRUD contract the class had always promised but never fully delivered. The original class docstring described full CRUD operations, but only create and read were implemented, so I added update and delete methods using MongoDB's update_many and delete_many, each returning a count of affected documents. On the security side, I fixed the way database credentials were handled: the original interpolated the username and password directly into the MongoDB connection URI with an f-string, which breaks whenever either value contains a reserved character such as an at-sign or a colon, so I now escape both with urllib.parse.quote_plus before they are placed in the connection string. I also made the error handling consistent. The original read method checked whether a collection was available before querying and failed gracefully if not, but create had no such guard and would have crashed with an unhandled exception on a failed connection, so I routed every method through a single connection-check helper. Finally, I added indexing on the fields the dashboard filters on most often, animal type, breed, and outcome type, so those queries stay efficient as the collection grows.

Course Outcomes

In my Module One plan, I identified this artifact as demonstrating secure and reliable data management, specifically schema and query considerations, credential security, and indexing. This milestone met that goal. The URL-encoding fix and the consistent connection guards both reflect a security mindset applied to the data layer, anticipating the inputs and failure conditions that would otherwise expose the system to crashes or malformed connection strings, and the

indexing work reflects attention to query performance rather than just correctness. I do not have major updates to my outcome-coverage plan. I would describe this enhancement as primarily demonstrating the databases portion of the well-founded techniques and tools outcome, with meaningful secondary coverage of the security mindset outcome, since credential handling and graceful failure are security concerns as much as database concerns.

Reflection

The most striking part of this process was that the artifact I planned to enhance was, in its original state, completely broken in a way that would have been embarrassing to discover during a job interview rather than during my own review. A corrupted file that will not even import is a worst-case version of the kind of thing a careful code review is supposed to catch, and finding it myself, well before it reached my ePortfolio, reinforced for me why reviewing your own past work critically is worth the time. It is easy to assume old coursework still runs; this was a direct reminder that assumption is not safe. The most valuable technical lesson came from testing the update method. I verified the enhanced module against a real in-memory MongoDB, and when I checked the number of documents my update reported changing, the count was lower than I expected. Rather than assume my code was wrong, I investigated and learned that MongoDB's modified_count deliberately excludes documents whose values did not actually change, so an update that sets a field to the value it already holds is correctly counted as zero modifications. That is a subtle but real detail of how the database behaves, and I would rather have learned it while testing my own code than been surprised by it in production. The main challenge throughout was resisting the urge to over-engineer. It would have been easy to add far more to this module than the dashboard actually needs, but I kept the enhancement focused on the

specific, justified improvements from my code review plan, completing the CRUD contract, securing the credentials, making failure handling consistent, and indexing the queried fields, rather than adding complexity for its own sake.